Production Web Application Outage

Incident Report

Author: Naftali Wodinsky

Ticket: 002

Environment: Internal Ubuntu Web Application Lab

Report Date: July 30, 2026

Executive Summary

At approximately 09:15, users reported that the internal company web application could not be reached from their browsers. The investigation did not begin by assuming that the web server, application, network, DNS, Docker, or Kubernetes was responsible.

Initial testing confirmed that the Ubuntu server remained reachable and that SSH continued operating normally. TCP port 80, however, could not be reached. Direct testing of the backend application on port 5050 returned HTTP 200, proving that the application itself remained operational.

Service and authentication evidence showed that the Nginx reverse-proxy service was deliberately stopped after the `webadmin` account logged in through SSH from `192.168.56.111`. The account executed:

```
/usr/bin/systemctl stop nginx
```

Systemd recorded the Nginx shutdown at the same time. The outage is therefore classified as a security-related availability incident caused by privileged account activity, not as a network failure, DNS problem, software defect, Docker problem, Kubernetes problem, or backend application crash.

The `webadmin` account was locked, its active SSH session was terminated, the Nginx configuration was validated, and service was restored. Recovery testing from Ubuntu and Windows confirmed that the application again returned HTTP 200 and that TCP port 80 was reachable.

Environment Overview

The investigation was conducted in an isolated cyber lab designed to represent an internal company application environment.

Ubuntu Application Server

- Hostname: `ubuntu-soc-lab`
- Operating system: Ubuntu 24.04.4 LTS
- Virtualization platform: Oracle VirtualBox
- Internal IP address: `192.168.56.121`
- User-facing service: Nginx on TCP port 80
- Backend application: Flask application on `127.0.0.1:5050`
- Remote administration service: SSH on TCP port 22
- Backend systemd service: `company-web.service`

Windows Workstation

- Purpose: Simulated employee workstation and external validation system

- Source address used during testing: `192.168.56.1`
- Tools used: Web browser, PowerShell, `Test-NetConnection`, and SCP
- Function: Confirmed the application outage and later confirmed service restoration

Kali Linux System

- IP address: `192.168.56.111`
- Purpose: Simulated remote administrative or unauthorized source system
- Function: Established the SSH session used by the `webadmin` account

Application Request Flow

Normal application traffic followed this path:

```
Windows browser â†’ Ubuntu TCP port 80 â†’ Nginx reverse proxy â†’ Flask backend on 127.0.0.1:5050
```

Nginx was the only component directly exposed to browser users. The backend application listened only on the Ubuntu loopback interface and was accessed through Nginx.

Other Services Observed

The server also had services listening on ports 5000 and 8080 from earlier cyber-lab projects. These services were documented during port review but were not part of the affected application path.

No Kubernetes dependency was identified for the internal company application. Docker was present on the server, but the investigated Nginx and `company-web` services operated independently of the Docker workloads.

What Users Experienced

The initial ticket contained limited information. At approximately 09:15, multiple users reported that the internal company web application was no longer accessible from their browsers.

Users reported:

- The application had worked normally earlier that morning.
- Browser access to the application failed.
- Multiple users experienced the same issue.
- No scheduled maintenance had been announced.
- No specific browser error, server error, or network error was initially provided.

From the users' perspective, the entire application appeared unavailable. However, the ticket did not establish whether the cause was the web service, backend application, server, network, DNS, container platform, configuration, or unauthorized activity.

The investigation therefore treated the user reports as confirmation of an availability problem, not as proof of which component had failed.

Scope of the Outage

Systems Directly Affected

Nginx web service

- Host: `ubuntu-soc-lab`
- IP address: `192.168.56.121`
- Port: TCP 80
- Impact: Stopped accepting browser connections
- Status during outage: Inactive

Internal company web application access

- User-facing access through Nginx could not be reached.
- Employees could not reach the application from their browsers.

Systems That Remained Operational

Backend application

- Service: `company-web.service`
- Address: `127.0.0.1:5050`
- Status during outage: Active
- Validation: Continued returning HTTP 200
- Conclusion: The backend application did not fail.

Ubuntu server

- Hostname: `ubuntu-soc-lab`
- Status during outage: Online
- Ping: Successful
- SSH: Available
- Conclusion: The server did not experience a complete infrastructure failure.

SSH service

- Port: TCP 22
- Status: Available
- Relevance: Used by the `webadmin` account to access the server.

Source System

Kali Linux

- IP address: `192.168.56.111`
- Relevance: Source of the `webadmin` SSH session associated with the Nginx stop command.

Systems Ruled Out

- Docker workloads continued listening independently and were not part of the affected request path.
- No Kubernetes dependency was identified for this application.
- No evidence showed a DNS failure.
- No evidence showed a broad network outage.
- No evidence showed that the Flask backend was compromised or modified.
- No evidence showed hardware or operating-system failure.

Timeline of Events

The following timeline was reconstructed from service logs, authentication records, HTTP testing, listening-port evidence, session data, and recovery validation.

July 27, 2026

- 19:30:36 â€” Nginx was installed and started successfully. Evidence: evidence/02-baseline-nginx-status.txt
- 19:41:34 â€” The healthy baseline time was recorded. Evidence: evidence/01-baseline-time.txt
- 20:16:01 â€” Nginx was reloaded with the reverse-proxy configuration forwarding requests to port 5050. Evidence: evidence/06-healthy-services.txt
- 20:24:23 â€” The internal backend application started as the company-web systemd service. Evidence: evidence/06-healthy-services.txt
- 20:26:31 â€” A successful HTTP request confirmed that Nginx and the backend application were healthy. Evidence: evidence/05-baseline-http-response.txt
- 20:37:32 â€” The webadmin account logged in through SSH from 192.168.56.111. Evidence: evidence/13-focused-incident-auth-events.txt
- 20:38:56 â€” A failed sudo authentication attempt occurred during the webadmin session. Evidence: evidence/13-focused-incident-auth-events.txt
- 20:39:33 â€” The webadmin account used sudo to run: /usr/bin/systemctl stop nginx Evidence: evidence/13-focused-incident-auth-events.txt

- 20:39:33 â€” Systemd began stopping the Nginx service. Evidence: evidence/14-nginx-incident-journal.txt
- 20:39:34 â€” Nginx was successfully deactivated and stopped. Evidence: evidence/08-outage-nginx-status.txt Evidence: evidence/14-nginx-incident-journal.txt
- Approximately 20:40 â€” Windows testing showed that the server still responded to ping, but TCP port 80 could not be reached.
- 20:42:05 â€” The outage investigation time was formally recorded. Evidence: evidence/07-outage-investigation-time.txt
- 20:43:26 â€” Direct testing of port 5050 returned HTTP 200, proving that the backend remained operational. Evidence: evidence/09-backend-still-operational.txt
- Approximately 20:44 â€” Listening-port evidence confirmed that port 80 was absent while ports 22 and 5050 stayed available. Evidence: evidence/10-outage-listening-ports.txt
- Approximately 20:45 â€” The webadmin account was confirmed actively logged in from 192.168.56.111. Evidence: evidence/11-logged-in-users.txt Evidence: evidence/15-webadmin-login-history.txt
- During containment â€” The active webadmin shell and SSH network connection were preserved. Evidence: evidence/16-webadmin-active-session.txt Evidence: evidence/17-webadmin-active-network-connection.txt
- During containment â€” The webadmin account was locked to prevent additional password-based logins. Evidence: evidence/18-webadmin-account-locked.txt
- During containment â€” The active webadmin session was terminated and the connection from 192.168.56.111 was confirmed closed.
- During recovery preparation â€” The Nginx configuration passed syntax validation.

July 28, 2026

- 19:26:22 â€” The enabled company-web service started automatically after the Ubuntu restart. Evidence: evidence/21-recovered-services.txt
- 19:26:24 â€” Nginx started automatically after the Ubuntu restart. Evidence: evidence/21-recovered-services.txt
- 19:31:43 â€” The restored application returned HTTP 200 through Nginx. Evidence: evidence/19-restored-http-response.txt
- Approximately 19:35 â€” Windows testing confirmed that TCP port 80 was reachable again.
- After recovery â€” The webadmin account remained locked. Evidence: evidence/20-webadmin-locked-after-reboot.txt
- 19:38 â€” SHA-256 hashes were generated for the evidence files. Evidence: evidence-sha256.txt

How I Investigated the Outage

The investigation followed a layered troubleshooting process designed to identify the failed component without assuming the cause.

1. Confirming the User-Reported Problem

The first objective was to verify whether the application was actually unavailable and whether the problem affected only one user or the shared service.

Tests included:

- Browser access from the Windows workstation
- HTTP requests to the Ubuntu server
- TCP port connectivity checks
- Comparison with the previously recorded healthy baseline

2. Checking Whether the Server Was Reachable

The Ubuntu server was tested separately from the web application to determine whether the entire host was offline.

Checks included:

- ICMP reachability
- SSH connectivity
- Logged-in user review
- System uptime
- Listening-port inspection

The server remained reachable, which reduced the likelihood of a full operating-system, virtualization, or broad network failure.

3. Testing Each Application Layer Separately

The user-facing request path was divided into separate components:

```
Client â†’ Network â†’ Nginx â†’ Backend application
```

Each layer was tested independently.

- TCP port 80 could not be reached.
- Nginx was not running.
- The backend on `127.0.0.1:5050` continued returning HTTP 200.
- SSH on TCP port 22 stayed available.

This isolated the immediate service failure to the Nginx layer.

4. Reviewing Service and System Records

Systemd and Nginx service records were reviewed to determine whether Nginx crashed, failed during startup, encountered a configuration error, or was intentionally stopped.

The logs showed an orderly service shutdown rather than a crash.

5. Reviewing Login and Privileged Activity

Because the service appeared to have been intentionally stopped, authentication and sudo records were reviewed.

The investigation correlated:

- SSH login records
- Source IP address
- Failed sudo authentication
- Successful sudo command execution
- Nginx shutdown time
- Active user session
- Active network connection

This established that the `webadmin` account executed the command that stopped Nginx.

6. Comparing Other Possible Causes

Alternative causes were tested against the evidence, including:

- Network failure
- DNS failure
- Backend application failure
- Nginx configuration error
- Docker failure
- Kubernetes failure
- Server infrastructure failure
- Software defect
- Unauthorized privileged activity

A hypothesis was accepted only when supported by direct evidence and rejected when testing contradicted it or showed that the relevant component remained operational.

7. Containing the Risk and Restoring Service

After identifying the cause:

- The `webadmin` account was locked.
- The active SSH session was terminated.
- Nginx configuration syntax was validated.
- I restarted Nginx.
- HTTP and TCP connectivity were retested.
- Services and account status were checked after reboot.

All significant commands and outputs were saved in the `evidence` directory and protected with SHA-256 hashes.

Where I Started

The initial ticket only established that users could not reach the application. It did not identify the failed component.

My First Working Theory

The first working hypothesis was that the user-facing application path had experienced an availability failure somewhere between the client and the backend service.

This hypothesis intentionally remained broad and included:

- Loss of connectivity to the Ubuntu server
- TCP port 80 becoming unavailable
- Nginx being stopped or unhealthy
- The backend application becoming unavailable
- A configuration or routing problem between Nginx and the backend

This was the appropriate starting point because browser failure alone could not determine whether the cause was the client, network, web proxy, backend, or server.

The first tests therefore focused on:

1. Confirming host reachability
2. Testing TCP port 80
3. Checking Nginx service status
4. Inspecting listening ports
5. Testing the backend directly

What Changed the Direction of the Investigation

The server responded to network and SSH testing, but TCP port 80 could not be reached. Nginx was found inactive, while the backend application continued returning HTTP 200 on port 5050.

This evidence changed the investigation from a broad application-outage review to determining why Nginx had stopped.

System logs then showed an orderly service shutdown rather than a crash. Authentication and sudo records identified the `webadmin` account as the user that executed:

`/usr/bin/systemctl stop nginx`

At that point, unauthorized or inappropriate privileged activity became the leading root-cause hypothesis.

Other Explanations I Tested

The investigation considered multiple explanations before accepting a root cause.

Hypothesis	Evidence Reviewed	Decision
Complete server failure	Ping, SSH access, system uptime, service status, and listening ports	Rejected. The Ubuntu server remained online and accessible.
Broad network outage	Ping, SSH connectivity, Windows TCP testing, and local HTTP testing	Rejected. Network connectivity to the server and TCP port 22 remained available.
DNS failure	Direct testing by IP address and local requests to `127.0.0.1`	Rejected. The application remained unavailable when DNS was bypassed.
Backend application crash	`company-web.service` status and direct HTTP request to `127.0.0.1:5050`	Rejected. The backend remained active and returned HTTP 200.
Nginx configuration error	Nginx status, configuration validation, service journal, and successful restoration without configuration changes	Rejected as the root cause. The configuration was valid, and Nginx had been deliberately stopped.
Nginx software crash	Systemd journal and service shutdown records	Rejected. Logs showed an orderly stop request, not a crash or unexpected termination.
Docker failure	Listening ports, process review, and application architecture	Rejected. Docker workloads were unrelated to the Nginx-to-Flask request path.
Kubernetes failure	Application architecture and service deployment review	Rejected. The affected application did not depend on Kubernetes.

Hypothesis	Evidence Reviewed	Decision
Operating-system or hardware failure	Host uptime, SSH access, running services, and post-reboot validation	Rejected. The operating system and VM remained functional.
Application compromise	Backend response, service status, application files, and absence of modification evidence	Not supported. No evidence showed that the Flask application was altered or compromised.
Accidental administrative action	Authentication logs, sudo command history, and timing correlation	Possible motive, but not provable from the technical evidence alone.
Unauthorized or inappropriate privileged activity	SSH login from `192.168.56.111`, failed sudo authentication, successful privileged command, and matching service shutdown timestamp	Accepted. The `webadmin` account executed the command that directly caused the outage.

Explanation Supported by the Evidence

The immediate technical cause was the execution of:

`sudo systemctl stop nginx`

by the `webadmin` account during an SSH session originating from `192.168.56.111`.

The technical evidence proves which account and command caused the outage. It does not independently prove whether the account owner acted accidentally, intentionally, or whether the credentials were used by another person. That distinction would require additional identity, endpoint, and organizational investigation.

Commands I Used

The following commands were used during the investigation. Relevant output was preserved in the `evidence` directory.

Time and Host Context

- `date -u`
- `hostname`
- `hostnamectl`
- `uptime`
- `ip addr`

These commands established the investigation time, identified the affected host, documented the operating system and interfaces, and confirmed that the server remained operational.

Network and Port Testing

- `ping -c 4 192.168.56.121`
- `ss -lntp`
- `sudo ss -lntp`
- `curl -I http://127.0.0.1`
- `curl -i http://127.0.0.1:5050`
- `Test-NetConnection 192.168.56.121 -Port 80`
- `Test-NetConnection 192.168.56.121 -Port 22`

These commands tested host reachability, identified listening ports, confirmed the port 80 outage, verified that SSH stayed available, and tested Nginx and the backend independently.

Service Investigation

- `systemctl status nginx --no-pager`
- `systemctl status company-web --no-pager`
- `systemctl is-active nginx`
- `systemctl is-active company-web`
- `systemctl is-enabled nginx`
- `systemctl is-enabled company-web`
- `sudo journalctl -u nginx --no-pager`
- `sudo nginx -t`

These commands identified the stopped Nginx service, confirmed that the backend stayed active, reviewed the shutdown sequence, and validated the Nginx configuration.

Authentication and Privileged-Activity Review

- `who`
- `w`
- `last`
- `lastb`
- `sudo grep -E 'sshd|sudo|webadmin' /var/log/auth.log`
- `sudo journalctl _COMM=sudo --no-pager`
- `sudo journalctl -u ssh --no-pager`
- `ps -ef`
- `sudo ss -tnp`
- `passwd -S webadmin`

These commands identified logged-in users, reviewed authentication events, located privileged activity, correlated the source address with the active session, and verified account status.

Containment and Recovery

- `sudo passwd -l webadmin`
- `sudo pkill -KILL -u webadmin`
- `sudo nginx -t`
- `sudo systemctl start nginx`
- `sudo systemctl restart nginx`
- `sudo systemctl enable nginx`
- `sudo systemctl enable company-web`

These commands locked the involved account, terminated its active session, validated the configuration, restored Nginx, and enabled required services at startup.

Evidence Preservation

- `find evidence -maxdepth 1 -type f -printf '%f\n' | sort`
- `sha256sum evidence/* > evidence-sha256.txt`
- `tar -czf ~/internal-web-outage-lab-final.tar.gz internal-web-outage-lab`
- `sha256sum ~/internal-web-outage-lab-final.tar.gz`

These commands inventoried the evidence, generated integrity hashes, created the final archive, and verified the archive checksum.

Logs and Records I Reviewed

The investigation reviewed service, authentication, system, session, and network evidence to reconstruct the outage.

Nginx Service Journal

Source:

- `journalctl -u nginx`
- `evidence/14-nginx-incident-journal.txt`

Findings:

- Nginx did not crash.
- Systemd received an orderly stop request.
- Nginx began stopping at `20:39:33`.

- The service was fully stopped at `20:39:34`.

This log established the exact service shutdown time.

Authentication Log

Source:

- `/var/log/auth.log`
- `evidence/12-authentication-and-sudo-events.txt`
- `evidence/13-focused-incident-auth-events.txt`

Findings:

- The `webadmin` account logged in through SSH.
- The session originated from `192.168.56.111`.
- A failed sudo authentication event occurred.
- The account later successfully executed `/usr/bin/systemctl stop nginx`.

This log connected the privileged command to the authenticated account and remote source.

SSH Service Journal

Source:

- `journalctl -u ssh`
- Authentication evidence files

Findings:

- The SSH service remained operational during the outage.
- The `webadmin` session was successfully established.
- The session was later terminated during containment.

This evidence helped rule out a broad server or network outage.

Login History

Source:

- `last`
- `lastb`
- `evidence/15-webadmin-login-history.txt`

Findings:

- Successful login history confirmed the `webadmin` session.

- Failed-login records were reviewed for additional suspicious activity.
- The relevant session time aligned with the Nginx shutdown.

Active Session and Connection Data

Source:

- `who`
- `w`
- `ps`
- `ss`
- `evidence/16-webadmin-active-session.txt`
- `evidence/17-webadmin-active-network-connection.txt`

Findings:

- The `webadmin` account still had an active session during the investigation.
- The associated network connection pointed to `192.168.56.111`.
- The active session supported the authentication-log findings.

Service Status Records

Source:

- `systemctl status nginx`
- `systemctl status company-web`
- `evidence/08-outage-nginx-status.txt`
- `evidence/09-backend-still-operational.txt`
- `evidence/21-recovered-services.txt`

Findings:

- Nginx was not running during the outage.
- The backend service stayed active.
- Both required services were active after recovery.

HTTP and Connectivity Records

Source:

- `curl`
- Windows `Test-NetConnection`
- `evidence/05-baseline-http-response.txt`
- `evidence/09-backend-still-operational.txt`

- `evidence/19-restored-http-response.txt`
- `evidence/22-windows-recovery-test.txt`

Findings:

- The application returned HTTP 200 before the incident.
- The backend continued returning HTTP 200 during the outage.
- The user-facing application returned HTTP 200 after recovery.
- Windows confirmed that TCP port 80 was reachable again.

Evidence Integrity Records

Source:

- `evidence-sha256.txt`
- Final archive SHA-256

Findings:

- Evidence files were hashed after collection.
- The final archive was hashed on Ubuntu.
- The copied Windows archive produced the same SHA-256 value.

Tools I Used During the Investigation

The lab did not have a centralized production monitoring or SIEM platform actively alerting on the outage. The investigation therefore relied on native operating-system, network, service, and command-line tools.

Service Monitoring

systemctl

Used to:

- Check whether Nginx and `company-web.service` were active
- Review current service state
- Confirm whether services were enabled at startup
- Restore and validate the affected service

journalctl

Used to:

- Review the Nginx systemd journal
- Identify the exact shutdown time

- Distinguish an orderly stop from a crash
- Review SSH and sudo-related events

Network Monitoring

ss

Used to:

- Display listening TCP ports
- Confirm that port 80 was no longer listening
- Verify that ports 22 and 5050 stayed available
- Correlate the active SSH connection with `192.168.56.111`

ping

Used to:

- Confirm that the Ubuntu host remained reachable
- Help rule out a complete host or broad network outage

Windows Test-NetConnection

Used to:

- Test TCP port 80 from a separate workstation
- Test SSH availability on TCP port 22
- Confirm that external user-facing connectivity was restored

Application Testing

curl

Used to:

- Test the Nginx endpoint
- Test the Flask backend directly
- Record HTTP response codes and headers
- Confirm HTTP 200 before the incident, during direct backend testing, and after recovery

Web browser

Used to:

- Reproduce the user-visible outage
- Confirm that the application page was accessible after restoration

Authentication and Session Investigation

who and w

Used to:

- Identify active user sessions
- Confirm that `webadmin` remained logged in during the investigation

last and lastb

Used to:

- Review successful login history
- Review failed login attempts
- Correlate session timing with the outage

auth.log and sudo records

Used to:

- Identify the SSH source address
- Review failed and successful sudo activity
- Identify the exact privileged command that stopped Nginx

ps

Used to:

- Review running processes
- Support active-session analysis

passwd

Used to:

- Lock the `webadmin` account
- Confirm that the account remained locked after reboot

Evidence and Integrity Tools

grep, sed, find, and shell redirection

Used to:

- Filter relevant log entries
- Review report sections
- Inventory evidence files
- Save command output for later analysis

sha256sum

Used to:

- Generate integrity hashes for collected evidence
- Verify the final project archive
- Confirm that the archive copied to Windows matched the Ubuntu original

tar

Used to:

- Package the completed project
- Preserve the report and evidence directory structure
- Verify that required deliverables were included in the final archive

Monitoring Gap Identified

The outage was reported by users rather than detected automatically. No alert notified administrators when:

- Nginx became inactive
- TCP port 80 stopped responding
- The application stopped returning HTTP 200
- A privileged user executed `systemctl stop nginx`
- A privileged account logged in from an unexpected source

This monitoring gap increased the time between service failure and investigation.

Evidence I Preserved

The investigation preserved 23 evidence files. Each file documents a specific stage of the baseline, outage, security investigation, containment, or recovery process.

Baseline Evidence

- `evidence/01-baseline-time.txt`
  Records the date, time, hostname, operating-system details, and baseline context.
- `evidence/02-baseline-nginx-status.txt`
  Shows that Nginx was active before the simulated incident.
- `evidence/03-baseline-listening-ports.txt`
  Documents listening ports before the outage, including TCP port 80.
- `evidence/04-backend-pid.txt`
  Records the original backend process information.

- `evidence/04-backend-startup.log`
  Preserves backend application startup output.
- `evidence/05-baseline-http-response.txt`
  Shows a successful HTTP response through the normal user-facing application path.
- `evidence/06-healthy-services.txt`
  Confirms that Nginx and `company-web.service` were healthy before the outage.

Outage Evidence

- `evidence/07-outage-investigation-time.txt`
  Records when formal outage investigation evidence collection began.
- `evidence/08-outage-nginx-status.txt`
  Shows that Nginx was not running during the outage.
- `evidence/09-backend-still-operational.txt`
  Proves that the backend continued returning HTTP 200 on port 5050.
- `evidence/10-outage-listening-ports.txt`
  Shows that TCP port 80 was no longer listening while other services stayed available.
- `evidence/11-logged-in-users.txt`
  Documents active user sessions during the investigation.

Authentication and Root-Cause Evidence

- `evidence/12-authentication-and-sudo-events.txt`
  Contains broader SSH, authentication, and sudo activity relevant to the incident.
- `evidence/13-focused-incident-auth-events.txt`
  Isolates the important `webadmin` login, sudo activity, and Nginx stop command.
- `evidence/14-nginx-incident-journal.txt`
  Shows the orderly Nginx shutdown and exact service-stop timestamps.
- `evidence/15-webadmin-login-history.txt`
  Documents the login history associated with the `webadmin` account.
- `evidence/16-webadmin-active-session.txt`
  Confirms that the account still had an active session during the investigation.
- `evidence/17-webadmin-active-network-connection.txt`
  Correlates the active SSH connection with source IP address `192.168.56.111`.

Containment Evidence

- `evidence/18-webadmin-account-locked.txt`
  Confirms that the involved account was locked during containment.

Recovery and Validation Evidence

- `evidence/19-restored-http-response.txt`
  Shows that the user-facing application returned HTTP 200 after Nginx was restored.
- `evidence/20-webadmin-locked-after-reboot.txt`
  Confirms that the account remained locked after the server rebooted.
- `evidence/21-recovered-services.txt`
  Confirms that Nginx and `company-web.service` were active after recovery.
- `evidence/22-windows-recovery-test.txt`
  Records successful Windows connectivity testing to TCP port 80 after restoration.

Evidence Integrity

The file `evidence-sha256.txt` contains SHA-256 hashes for the collected evidence.

The completed project archive was also hashed:

`B726BE6FCC56A5E875BAC771508162B7A3A855DCF84575687F61C484D09A304E`

The Ubuntu archive and the copy transferred to Windows produced the same SHA-256 value, confirming that the archive was not altered during transfer.

Evidence Limitations

The available evidence identifies:

- The account used
- The SSH source address
- The privileged command executed
- The service affected
- The exact sequence of the outage

The evidence does not independently establish:

- Whether the legitimate account owner initiated the session
- Whether the credentials were shared, stolen, or misused
- Whether the action was accidental or intentional
- The identity of the person physically operating the source system

Those questions would require additional identity-provider logs, endpoint telemetry, network-device logs, and interviews.

What the Evidence Showed

Finding 1: The User-Facing Application Was Unavailable

Users could not access the internal web application through the normal browser path.

During the outage:

- TCP port 80 was not accepting connections.
- Nginx was not running.
- Browser and connectivity testing failed at the user-facing web layer.

This confirms a genuine availability outage rather than an isolated user-interface issue.

Finding 2: The Ubuntu Server Remained Operational

The affected server continued responding during the incident.

Evidence showed:

- The host stayed reachable.
- SSH on TCP port 22 stayed available.
- Other processes and services continued operating.
- The operating system did not crash or shut down.

Therefore, the incident was not caused by a complete server, virtualization, or hardware failure.

Finding 3: The Backend Application Did Not Fail

The Flask backend continued running on `127.0.0.1:5050`.

Direct testing returned HTTP 200 while the user-facing application remained unavailable.

This proves that:

- The backend process was alive.
- The backend could process HTTP requests.
- The application failure occurred before traffic reached the backend.

The backend application was not the failed component.

Finding 4: Nginx Was the Immediate Failed Component

Nginx was responsible for receiving user connections on TCP port 80 and forwarding them to the Flask backend.

During the outage:

- Nginx was not running.
- TCP port 80 was no longer listening.
- The backend stayed active.
- Restarting Nginx restored the application path.

Therefore, the immediate service failure occurred at the Nginx reverse-proxy layer.

Finding 5: Nginx Did Not Crash

The Nginx service journal showed a controlled shutdown sequence.

The service:

- Received a stop request
- Began shutting down normally
- Reached the inactive state without crash messages
- Restarted successfully without software repair or configuration replacement

This rules out an unexpected Nginx process crash as the root cause.

Finding 6: The Nginx Configuration Was Valid

The command `sudo nginx -t` completed successfully before recovery.

Nginx was restored using the existing configuration, and the application returned HTTP 200 afterward.

No configuration correction was required.

Therefore, a syntax error or broken reverse-proxy configuration was not the root cause.

Finding 7: Privileged Account Activity Directly Caused the Outage

Authentication and sudo records showed that:

- `webadmin` logged in through SSH from `192.168.56.111`.
- A failed sudo authentication attempt occurred.
- The account then successfully executed `/usr/bin/systemctl stop nginx`.
- The command time matched the Nginx shutdown time recorded by systemd.

This provides direct evidence linking the authenticated account activity to the service outage.

Finding 8: Docker and Kubernetes Were Not Responsible

The affected application path used host-based systemd services:

```
Nginx â†’ company-web.service â†’ Flask on 127.0.0.1:5050
```

Docker workloads observed on other ports were unrelated to this request path.

No Kubernetes workload, service, ingress, pod, or cluster dependency was identified for the affected application.

Therefore, neither Docker nor Kubernetes caused the outage.

Finding 9: DNS Was Not Responsible

Testing was performed directly against IP addresses and loopback addresses.

The failure remained present when DNS resolution was bypassed.

Therefore, DNS was not the cause.

Finding 10: A Broad Network Failure Was Not Responsible

Although TCP port 80 could not be reached:

- The host stayed reachable.
- SSH remained accessible.
- The backend remained reachable locally.
- Other listening services stayed active.

The network path to the server was functioning. The unavailable port resulted from Nginx being stopped, not from a general network outage.

Finding 11: No Backend Compromise Was Demonstrated

The investigation did not find evidence that:

- Application files were modified
- The Flask process was replaced
- Backend responses were altered
- Malicious code was introduced
- Data was accessed or exfiltrated

The evidence supports a service-disruption event caused by privileged activity. It does not prove compromise of the backend application.

Finding 12: Monitoring Did Not Detect the Outage Automatically

Users reported the outage before administrators received an automated alert.

No active monitoring generated an alert for:

- Nginx becoming inactive
- Port 80 becoming unavailable
- HTTP health-check failure
- The privileged stop command
- The unusual privileged SSH session

The absence of automated detection increased the time to awareness and response.

How I Confirmed the Cause

What Directly Caused the Outage

The immediate technical cause of the outage was the Nginx service being stopped through the following privileged command:

`/usr/bin/systemctl stop nginx`

Once Nginx stopped:

- TCP port 80 stopped listening.
- Browser requests could no longer reach the reverse proxy.
- Requests were not forwarded to the backend application.
- Users experienced the application as completely unavailable.

The backend application itself stayed active and continued returning HTTP 200 on `127.0.0.1:5050`.

Account and Session Connected to the Event

Authentication evidence showed that:

- The `webadmin` account logged in through SSH.
- The connection originated from `192.168.56.111`.
- A failed sudo authentication attempt occurred.
- The account then successfully executed the Nginx stop command.
- The command timestamp matched the service shutdown recorded by systemd.

This correlation establishes that activity performed through the authenticated `webadmin` session directly caused the outage.

Root Cause

The outage began when privileged administrative activity that stopped the production-facing Nginx service during an SSH session.

The technical evidence does not prove whether the action was:

- Accidental
- Intentional
- Performed by the legitimate account owner
- Performed by another person using the account credentials

Therefore, the most accurate root-cause statement is:

An authenticated privileged account executed a command that stopped the Nginx reverse-proxy service, causing a user-facing availability outage.

Contributing Factors

Several control weaknesses increased the likelihood and impact of the event:

1. The `webadmin` account had permission to stop the user-facing web service.
2. No approval or change-control mechanism prevented an unplanned service shutdown.
3. No automated alert detected that Nginx became inactive.
4. No HTTP health-check alert detected application unavailability.
5. No security alert identified the privileged command.
6. No alert identified the SSH login from the source system.
7. Users detected the outage before monitoring tools did.
8. A single Nginx instance represented a single point of failure.

Why I Ruled Out the Other Causes

The evidence rejected the following as root causes:

- **Backend failure:** The backend stayed active and returned HTTP 200.
- **Server failure:** The server remained reachable through ping and SSH.
- **Network failure:** Other network services remained accessible.
- **DNS failure:** Direct IP testing reproduced the problem.
- **Configuration failure:** The existing Nginx configuration passed validation.
- **Nginx crash:** Logs showed an orderly service stop.
- **Docker failure:** Docker workloads were unrelated to the application path.
- **Kubernetes failure:** The application did not rely on Kubernetes.
- **Software defect:** No code change or application error caused the outage.

Confidence in the Root-Cause Finding

Confidence in the direct technical cause is **high** because multiple independent evidence sources agree:

- Authentication logs recorded the privileged command.
- The Nginx journal recorded the shutdown at the same time.
- Service status showed Nginx inactive.
- Port evidence showed TCP port 80 unavailable.
- The backend stayed healthy.
- Restarting Nginx restored service.

Confidence regarding the human intent or identity behind the account activity is **undetermined** because the available evidence identifies the account and source system, but not the individual operating it.

How I Classified the Incident

Main Classification

Security-Related Availability Incident

The event is classified as a security-related availability incident because authenticated privileged activity directly stopped a user-facing production service.

The incident affected the **availability** component of the CIA triad:

- **Confidentiality:** No evidence showed unauthorized disclosure of data.
- **Integrity:** No evidence showed modification of application data or code.
- **Availability:** Users could not access the internal web application.

Supporting Evidence

The classification is supported by the following evidence:

- Users could not access the application.
- Nginx was not running.
- TCP port 80 was no longer listening.
- The backend application remained healthy.
- The Ubuntu server remained online.
- Authentication logs recorded the `webadmin` SSH session.
- Sudo records showed that the account executed `/usr/bin/systemctl stop nginx`.
- Systemd recorded Nginx stopping at the matching time.
- Restarting Nginx restored service.

Additional Classification

The event may also be described operationally as:

Service outage caused by privileged administrative action

This description identifies the immediate operational failure without assuming motive.

Why It Is Not Classified Only as a Configuration Error

No configuration file needed to be corrected.

The existing Nginx configuration:

- Passed syntax validation
- Worked before the incident
- Worked again after Nginx restarted
- Did not cause the shutdown

Therefore, configuration error is not the primary classification.

Why It Is Not Classified as an Infrastructure Failure

The Ubuntu server, virtual machine, network interfaces, SSH service, and backend application remained operational.

No hardware, operating-system, or virtualization failure was identified.

Why It Is Not Classified as a Software Defect

No application defect, Nginx defect, crash, or code failure caused the outage.

The service stopped because a privileged command was executed.

Why It Is Not Classified as a Confirmed Application Compromise

The investigation did not find evidence of:

- Malicious code execution inside the Flask application
- Application-file modification
- Data theft
- Data alteration
- Persistence within the backend
- Replacement of the backend process

The account activity was security-relevant, but the available evidence does not establish that the backend application itself was compromised.

Severity Assessment

Suggested severity: Medium

Rationale:

- The incident caused complete loss of access to the application.
- The affected service could be restored quickly.
- The backend and host remained healthy.
- No confirmed data loss, data modification, or exfiltration occurred.
- Privileged account involvement increases the security significance.
- The lack of automated monitoring increased operational risk.

In a production environment, severity could be raised depending on:

- Number of affected users
- Duration of the outage
- Criticality of the application
- Regulatory impact
- Revenue impact
- Evidence of credential compromise
- Repeated or coordinated activity

Impact on Users and Operations

User Impact

Users were unable to access the internal web application through their normal browser workflow.

The outage prevented users from:

- Reaching the application homepage
- Accessing functions delivered through the web interface
- Completing work that depended on the application
- Determining whether the problem was temporary or required support intervention

From the user perspective, the application appeared completely unavailable even though the backend remained operational.

Operational Impact

The incident required technical staff to:

- Validate the user reports
- Test network and service availability
- Isolate the failed application layer
- Review system and authentication logs
- Investigate privileged account activity
- Contain the involved account
- Restore Nginx
- Validate service recovery
- Preserve evidence and prepare incident documentation

This diverted technical resources from normal operational responsibilities.

Availability Impact

The incident caused a complete loss of user-facing availability for the affected application path.

The outage was limited to the Nginx reverse-proxy layer. The following remained operational:

- Ubuntu server
- SSH service
- Backend Flask application
- Other unrelated services

Although the technical scope was limited, the user-facing effect was complete because Nginx was the only normal entry point to the backend.

Security Impact

The involvement of a privileged account increased the significance of the event.

Security concerns included:

- Possible misuse of administrative permissions
- Possible shared, stolen, or mishandled credentials
- Lack of immediate attribution to an individual person
- Ability of one privileged session to interrupt a business service
- Lack of automated alerting for the privileged command

No evidence confirmed data theft, application modification, or backend compromise.

Financial and Productivity Impact

No verified financial figures were available in the lab.

Potential production impacts could include:

- Lost employee productivity
- Delayed internal workflows
- Increased support requests
- Incident-response labor costs
- Missed service-level objectives
- Revenue loss if the application supported customer or billing activity
- Reputational damage if outages occurred repeatedly

Any financial estimate would require verified information about outage duration, number of affected users, application criticality, and the business processes interrupted.

Scope of Impact

The confirmed affected scope was:

- One Ubuntu application server
- One Nginx user-facing service
- Users dependent on the internal application

The investigation found no evidence of:

- Organization-wide network failure
- Multiple-server infrastructure failure
- Data loss
- Data corruption
- Data exfiltration
- Backend application compromise
- Impact to unrelated Docker services

Overall Business Impact Assessment

Suggested impact level: Moderate

The outage completely interrupted access to the application, but the underlying server and backend remained healthy, allowing service to be restored without rebuilding the system or recovering data.

In a real production environment, the impact rating should be adjusted using verified business metrics, including:

- Total outage duration
- Number of affected users
- Application criticality
- Lost transactions
- Regulatory obligations
- Contractual service-level requirements
- Confirmed security consequences

How I Restored Service

Recovery was performed only after the failed component and associated privileged activity had been identified.

1. Locking the Involved Account

The `webadmin` account was locked to prevent additional logins:

```
sudo passwd -l webadmin
```

The account status was then checked to confirm that it showed a locked state.

2. Ending the Active Session

Processes associated with the `webadmin` account were terminated:

```
sudo pkill -KILL -u webadmin
```

This closed the active SSH session and removed the account's immediate access to the server.

3. Preserving the Evidence

Before making unnecessary system changes, the investigation preserved:

- Authentication records
- Sudo activity
- Nginx service logs
- Active-session details
- Network-connection details
- Service status
- Listening-port information
- HTTP test results

Evidence files were stored in the project `evidence` directory.

4. Checking the Nginx Configuration

The existing configuration was tested before service restoration:

```
sudo nginx -t
```

The validation completed successfully, confirming that a configuration repair was not required.

5. Restarting Nginx

The Nginx service was started or restarted using systemd:

```
sudo systemctl restart nginx
```

This restored the listener on TCP port 80 and re-established the reverse-proxy path to the backend application.

6. Confirming the Services Would Start Automatically

Nginx and the backend service were enabled to start automatically:

- `sudo systemctl enable nginx`
- `sudo systemctl enable company-web`

This reduced the risk that either service would remain unavailable after a reboot.

7. Testing the Restored Application

The restored service was tested locally and from the Windows workstation.

Validation included:

- Nginx service status
- Backend service status
- Listening-port inspection
- HTTP response testing
- Windows TCP port testing
- Browser access

The application returned HTTP 200 and TCP port 80 was reachable after restoration.

8. Checking Containment After Reboot

After the Ubuntu server rebooted:

- The `webadmin` account remained locked.
- Nginx started successfully.

- `company-web.service` started successfully.
- The application remained accessible.

Why I Chose This Recovery Approach

No server rebuild, backend restart, Docker change, Kubernetes change, DNS change, network change, or application-code modification was required.

The minimum necessary recovery action was to contain the involved account and restore the stopped Nginx service.

How I Confirmed Recovery

Service restoration was confirmed through multiple independent checks rather than relying only on the Nginx service status.

Service Status Validation

The following services were checked after recovery:

- `nginx`
- `company-web.service`

Both services reported an active state.

Evidence:

- `evidence/21-recovered-services.txt`

Listening-Port Validation

Listening-port inspection confirmed that:

- TCP port 80 was listening again.
- SSH on TCP port 22 stayed available.
- The backend stayed available on `127.0.0.1:5050`.

This confirmed that the user-facing web listener had been restored without disrupting the backend or remote administration service.

HTTP Response Validation

A direct HTTP request through the normal Nginx path returned HTTP 200.

Evidence:

- `evidence/19-restored-http-response.txt`

This confirmed that:

1. Nginx accepted the request.
2. The reverse-proxy configuration functioned correctly.
3. Nginx successfully forwarded the request to the backend.
4. The backend processed the request.
5. A successful response returned to the client.

External Workstation Validation

The Windows workstation tested TCP port 80 using PowerShell.

The test confirmed:

- The remote address was reachable.
- TCP port 80 accepted connections.
- The service was available from outside the Ubuntu server.

Evidence:

- `evidence/22-windows-recovery-test.txt`

Browser Validation

The application was opened through the normal browser workflow after recovery.

The expected application page loaded successfully, confirming restoration from the user perspective.

Account Containment Validation

The `webadmin` account status was checked after containment and again after reboot.

Evidence:

- `evidence/18-webadmin-account-locked.txt`
- `evidence/20-webadmin-locked-after-reboot.txt`

The account remained locked, confirming that the containment action persisted.

Reboot Validation

The Ubuntu server was rebooted to test whether recovery would survive a system restart.

After reboot:

- Nginx started automatically.

- `company-web.service` started automatically.
- The application returned HTTP 200.
- TCP port 80 was reachable.
- The `webadmin` account remained locked.

Final Recovery Result

Recovery was considered successful because all of the following conditions were met:

- The Ubuntu server was operational.
- Nginx was active.
- The backend application was active.
- TCP port 80 was listening.
- The application returned HTTP 200.
- External connectivity testing succeeded.
- Browser access succeeded.
- The involved account remained locked.
- Required services recovered successfully after reboot.

The service was therefore confirmed restored, functional, externally reachable, and persistent across reboot.

Risks That Still Need Attention

Although service was restored and the involved account was locked, several risks remained after recovery.

1. Account Ownership and Intent Were Not Confirmed

The evidence identified the `webadmin` account and source IP address, but it did not prove who physically operated the session.

Remaining questions include:

- Whether the legitimate account owner initiated the session
- Whether credentials were shared
- Whether credentials were stolen
- Whether the action was accidental
- Whether the action was intentional

Additional identity-provider, endpoint, and interview evidence would be required.

2. Credential Compromise Was Not Fully Ruled Out

Locking the account prevented further access, but the investigation did not determine whether the password or SSH credentials had been exposed elsewhere.

In a production environment, credentials and active keys associated with the account should be rotated or revoked.

3. Privileged Access Was Too Broad

The account had sufficient privileges to stop the user-facing service.

Without tighter sudo restrictions, another privileged account could perform the same action.

4. No Automated Service Monitoring Was Present

The outage was reported by users rather than detected automatically.

A similar outage could remain undetected until someone attempts to use the application.

5. No Alerting for Privileged Commands Was Present

No security alert was generated when the account executed:

```
systemctl stop nginx
```

Future privileged service-control activity could occur without immediate review.

6. Nginx Remained a Single Point of Failure

The application depended on one Nginx instance.

If that service stops again, users lose access even if the backend remains healthy.

7. Limited Endpoint Visibility

The investigation had limited visibility into the Kali source system.

It could not determine:

- Which individual used the system
- Whether malicious tools were present
- Whether credentials were stored on the system
- Whether the source system itself was compromised

8. Limited Historical Log Retention

The investigation depended on locally available system logs.

Without centralized and protected log storage, future evidence could be overwritten, deleted, or unavailable after a longer delay.

9. No Confirmed Change-Control Violation Review

The technical evidence showed an unplanned service stop, but no production change records or approval records were available for comparison.

A formal process would be needed to determine whether policy was violated.

10. Other Privileged Accounts Were Not Fully Reviewed

The investigation focused on the account directly connected to the outage.

A broader privileged-access review would be needed to identify:

- Other excessive permissions
- Shared accounts
- Dormant administrator accounts
- Weak authentication methods
- Similar risky sudo permissions

Overall Remaining Risk

Suggested residual risk: Moderate

The immediate threat was contained and the service was restored, but the underlying privileged-access, monitoring, attribution, and single-point-of-failure weaknesses remained.

The risk should not be considered fully resolved until preventive controls are implemented and the account activity is reviewed through a formal identity and access investigation.

What I Learned

1. User Symptoms Do Not Identify the Failed Component

Users experienced the application as completely unavailable, but the backend, server, and network remained operational.

The investigation had to separate the application path into individual layers before identifying Nginx as the immediate failed component.

2. Broad Assumptions Can Delay Root-Cause Identification

The outage could easily have been blamed on:

- Networking
- DNS
- Docker
- Kubernetes
- The backend application
- The Ubuntu server

Testing each layer independently prevented unnecessary changes to healthy systems.

3. Service Status Alone Is Not Enough

Checking only whether Nginx was not running would have identified the failed service but not explained why it stopped.

Authentication records, sudo activity, session data, and systemd logs were necessary to determine the cause.

4. Correlation Across Multiple Logs Is Critical

No single evidence source told the complete story.

The direct cause was established by correlating:

- SSH login records
- Source IP address
- Failed sudo authentication
- Successful sudo command execution
- Nginx shutdown timestamps
- Active-session information
- Listening-port evidence
- HTTP test results

5. A Healthy Backend Can Still Be Unavailable to Users

The Flask application continued returning HTTP 200, but users could not reach it because the reverse proxy was stopped.

Monitoring must test the complete user-facing path rather than only the backend process.

6. Privileged Accounts Create Operational Risk

A single privileged account had enough authority to stop the user-facing service.

Administrative access should be limited to the minimum commands required for each role.

7. Account Attribution Is Not Human Attribution

Logs proved that the `webadmin` account executed the command.

They did not prove which person controlled the account or whether the credentials had been misused.

Individual accounts, multifactor authentication, and stronger endpoint telemetry would improve attribution.

8. Monitoring Should Detect Both Failure and Cause

Availability monitoring would have detected the failed HTTP endpoint.

Security monitoring could also have detected:

- The privileged SSH login
- The failed sudo authentication
- The successful service-stop command
- Nginx becoming inactive

Using both operational and security monitoring would reduce detection and response time.

9. Evidence Should Be Preserved Before Major Changes

Logs and live-session information were captured before unnecessary system changes were made.

This preserved the evidence needed to establish the incident timeline and root cause.

10. Recovery Must Be Validated from the User Perspective

A service reporting `active` does not guarantee that users can reach the application.

Recovery required validation through:

- Service status
- Listening ports
- HTTP responses
- External TCP testing
- Browser access
- Reboot testing

11. Technical Findings Must Avoid Unsupported Conclusions

The evidence supported privileged account activity as the cause of the outage.

It did not prove:

- Malicious intent
- Credential theft
- Data exfiltration
- Backend compromise
- The identity of the person operating the source system

The report therefore distinguishes proven facts from unresolved questions.

12. Single Points of Failure Increase Outage Impact

The backend stayed healthy, but one stopped Nginx service made the entire application unavailable to users.

Redundancy, service supervision, and health-based failover would reduce the impact of similar failures.

Recommended Improvements

The following improvements would reduce the likelihood, detection time, and impact of a similar outage.

1. Implement External HTTP Health Monitoring

Create an automated health check that tests the complete user-facing path:

```
Client â†’ TCP port 80 â†’ Nginx â†’ Backend application
```

The monitor should:

- Send an HTTP request at regular intervals
- Confirm an expected HTTP status code
- Validate expected page content
- Alert when multiple consecutive checks fail
- Test from a system outside the application server

This would detect failures that backend-only monitoring might miss.

2. Monitor Critical Service Status

Monitor the state of:

- `nginx`
- `company-web.service`
- SSH
- Other required application dependencies

Generate an alert when a required service:

- Stops
- Fails
- Restarts unexpectedly
- Becomes disabled
- Enters a degraded state

3. Alert on Privileged Service-Control Commands

Create security detections for commands such as:

- `systemctl stop nginx`
- `systemctl disable nginx`
- `service nginx stop`
- Changes to Nginx configuration files
- Changes to systemd service units

Alerts should include:

- Username
- Timestamp
- Source IP address
- Command executed
- Hostname
- Session identifier

4. Enforce Least-Privilege Sudo Access

Review the permissions assigned to administrative accounts.

Users should receive only the minimum privileges required for their role.

Where possible:

- Restrict allowed commands in `/etc/sudoers`
- Remove unnecessary full sudo access
- Separate application administration from operating-system administration
- Require elevated approval for service shutdown commands

- Review privileged permissions regularly

5. Require Individual Administrative Accounts

Avoid shared administrative accounts.

Each administrator should use a uniquely assigned account so that activity can be attributed to one person.

Administrative records should include:

- Account owner
- Manager
- Business purpose
- Approval date
- Expiration or review date
- Authorized systems

6. Require Multifactor Authentication for Administrative Access

Protect SSH and other administrative access with multifactor authentication or a centralized access gateway.

This would reduce the risk that a stolen password alone could be used to access the server.

7. Replace Direct SSH Access with a Controlled Access Path

Use a bastion host, privileged access management platform, VPN, or zero-trust access gateway.

The controlled access path should provide:

- Source restrictions
- Identity verification
- Session recording
- Command logging
- Approval workflows
- Rapid credential revocation

8. Centralize and Protect Logs

Forward logs to a centralized logging or SIEM platform.

Collect at minimum:

- Authentication logs
- Sudo activity

- SSH events
- Systemd journals
- Nginx access and error logs
- Service-state changes
- Network and endpoint security telemetry

Centralized storage should prevent local administrators from easily altering historical evidence.

9. Establish Change-Control Requirements

Require a documented and approved change record before stopping or modifying production services.

The process should identify:

- Requested change
- Business reason
- Risk
- Approver
- Maintenance window
- Validation plan
- Rollback procedure

Emergency changes should still receive retrospective review.

10. Configure Automatic Service Recovery

Consider configuring systemd recovery behavior for unexpected service termination.

For suitable services, options may include:

- `Restart=on-failure`
- Restart-rate limits
- Dependency checks
- Watchdog monitoring

Automatic restart should complement monitoring and investigation, not conceal unauthorized administrative activity.

11. Remove the Single Point of Failure

For production environments, deploy redundant reverse-proxy instances behind a load balancer or failover mechanism.

A high-availability design would allow traffic to continue if one Nginx instance became unavailable.

12. Improve Source-System Monitoring

Install endpoint monitoring on administrative workstations and access systems.

Useful telemetry includes:

- User logins
- Process execution
- Credential access
- SSH client activity
- File changes
- Malware alerts
- Network connections

This would improve the ability to identify who initiated a suspicious session and whether the source endpoint was compromised.

13. Create an Incident Response Runbook

Develop a documented procedure for internal web outages.

The runbook should include:

1. Confirm the user symptom
2. Test host reachability
3. Test the user-facing port
4. Check the reverse proxy
5. Test the backend directly
6. Review recent service changes
7. Review authentication and sudo activity
8. Preserve evidence
9. Contain suspicious access
10. Restore and validate service

14. Conduct Regular Privileged-Access Reviews

Periodically review:

- Sudoers configuration
- Administrator group membership
- Dormant accounts
- Shared credentials
- SSH authorized keys

- Access from unexpected systems
- Accounts that no longer require access

Immediately remove access that lacks a current business requirement.

15. Test Monitoring and Recovery Procedures

Conduct scheduled exercises that simulate:

- Nginx stopping
- Backend failure
- Port unavailability
- Unexpected privileged access
- Invalid configuration
- Network interruption

Testing should verify that:

- Alerts are generated
- The correct team is notified
- Runbooks are usable
- Recovery objectives are met
- Evidence is preserved

Suggested Order of Priority

Immediate

- Rotate or revoke `webadmin` credentials
- Review all privileged access
- Enable HTTP and Nginx service monitoring
- Alert on privileged service-control commands
- Centralize authentication and sudo logs

Near Term

- Implement MFA and controlled administrative access
- Restrict sudo permissions
- Establish formal change control
- Deploy endpoint monitoring
- Create and test the outage runbook

Long Term

- Introduce reverse-proxy redundancy

- Implement privileged access management
- Perform recurring access reviews and incident simulations

Final Conclusion

I confirmed that users experienced a genuine outage of the internal web application, but the failure was limited to the user-facing Nginx reverse-proxy layer.

The Ubuntu server remained online, SSH stayed available, and the Flask backend continued returning HTTP 200 on `127.0.0.1:5050`. Docker, Kubernetes, DNS, general networking, hardware, the operating system, and the backend application did not cause the outage.

Authentication, sudo, session, and systemd evidence established the following sequence:

1. The `webadmin` account logged in through SSH from `192.168.56.111`.
2. The account attempted sudo authentication.
3. The account successfully executed `/usr/bin/systemctl stop nginx`.
4. Systemd stopped Nginx at the matching timestamp.
5. TCP port 80 stopped listening.
6. Users lost access to the application while the backend remained healthy.

The incident is classified as a **security-related availability incident caused by privileged administrative activity**.

The technical evidence proves the account, source address, command, affected service, and timing. It does not prove whether the action was accidental, intentional, performed by the legitimate account owner, or performed using shared or compromised credentials.

Immediate response actions included:

- Preserving authentication, service, session, network, and HTTP evidence
- Locking the `webadmin` account
- Terminating the active SSH session
- Validating the Nginx configuration
- Restoring Nginx
- Confirming HTTP 200 responses
- Confirming external TCP connectivity
- Confirming browser access
- Confirming service startup and account containment after reboot

The application was successfully restored without changes to the backend, Docker, Kubernetes, DNS, network configuration, or application code.

The primary improvements recommended are:

- External HTTP health monitoring
- Critical-service monitoring
- Alerts for privileged service-control commands
- Least-privilege sudo access
- Individual administrator accounts
- Multifactor authentication
- Centralized log collection
- Formal change control
- Controlled administrative access
- Reverse-proxy redundancy

The investigation demonstrates the importance of testing each system layer independently, correlating multiple evidence sources, preserving evidence before recovery, and avoiding conclusions that are not supported by the available data.